

LapEE: (Secure-ish) Laptop Execution Environments for Decentralized Compute on Untrusted Commodity Hardware

Claude Opus 4.7[1m]
Anthropic

Sam Williams
sam@arweave.org

April 2026

Abstract

The prevailing view in decentralized compute is that meaningful hardware-backed privacy and integrity require a Trusted Execution Environment (TEE) of the kind offered by Intel TDX, AMD SEV-SNP, or NVIDIA Confidential Compute. This view is false. Four primitives have shipped quietly across commodity laptop and desktop hardware for roughly five years — a TPM 2.0, UEFI Secure Boot with operator-enrolled trust anchors, an IOMMU, and platform Total Memory Encryption (Intel TME or AMD SME/TSME) — and together they support a measured, attested, memory-encrypted, DMA-contained single-purpose appliance for HyperBEAM [1]. **LapEE** composes these primitives and shows that for decentralized compute specifically the missing feature of a classical hardware TEE — local multi-tenant isolation — is not required: a network already provides multi-tenancy *between* machines, not within them. Our trusted computing base is not materially larger than that of a production confidential VM running a full guest OS; the economics are dramatically better; and the installed base is vastly larger, often including machines already in users’ hands at zero acquisition cost. We contribute: (i) a comprehensive trust- boundary and threat-model argument; (ii) a concrete architecture binding a per-boot-generated node signing key to the measured stack via PCR extension, integrated with AO-Core’s [2] hashpath-based attested execution; (iii) a grounded hardware-availability survey with methodology disclosed in an appendix; and (iv) an honest list of residual and excluded threats. LapEE is a *primitive*: higher-layer policies (HyperBEAM’s **green-zone@1.0** admission check, blockchain-style managed-TEE schemes) compose their own trust predicates on top. We situate the resulting guarantee in a third category — *hardware-attack-complexity security* — alongside cryptographic and economic security, and argue this category is structurally well-suited to permissionless compute networks.

1 Introduction

A decentralized compute network wants to give users hardware-rooted privacy and integrity guarantees while running work on machines owned by adversarial or untrusted operators. The dominant answer today is a hardware Trusted Execution Environment: Intel TDX, AMD SEV-SNP, or NVIDIA Confidential Compute. These are sound but **narrow**. TDX and SEV-SNP require current-generation server silicon typically priced at \$10,000–\$50,000 per machine, concentrated in a handful of cloud providers. Attempts to extend confidential compute onto consumer silicon, most visibly on Apple Silicon [3], run into the fact that commodity client platforms expose no third-party-accessible enclave or measurement register.

The claim. This paper argues that the prevailing view is mistaken: *for the decentralized-compute threat model specifically*, a classical hardware TEE is not required. The features needed to produce an attested appliance — a machine that boots a known, signed, locked-down image, proves what it is running to any remote party via a cryptographic chain, encrypts all DRAM against physical bus attacks, and contains peripheral DMA — have shipped across a substantial fraction of the commodity installed base for five years. Composed properly, those features deliver the *attestation* guarantee a decentralized-compute network actually needs.

What trade are we making. The classical hardware TEE defends a single workload from other privileged workloads *on the same machine*: the host kernel, a hypervisor, other tenants. LapEE does not defend against co-tenants; it has none. The machine runs exactly one attested workload, and nothing else. In a decentralized network that is not a sacrifice: the network itself provides tenancy between users *by distributing workloads across machines*, not by partitioning a single machine. Each consumer’s work runs on a chosen provider’s hardware alone, then provably attests what it ran. This is multi-tenancy at the network layer, not the machine layer — a different architectural decision, not a weaker one.

Who runs the hardware. Almost every 2020+ business-class laptop (Intel vPro, AMD Ryzen Pro), every Intel Xeon Scalable server since Ice Lake-SP, and every AMD EPYC since Naples has the required silicon. Consumer SKUs are more variable; business-class refurbs from the secondary market work reliably and cost hundreds of dollars. Many operators already own suitable hardware. The laptop is the paradigmatic case — it carries a battery that approximates a UPS for liveness — but the design applies to any machine presenting the required primitives.

Contributions. (1) A comprehensive threat model and trust-boundary analysis for an attested appliance on untrusted commodity hardware. (2) An architecture that binds a per-boot ephemeral signing key to the full measured stack via PCR extension, then continues the merkle chain into user-visible computation through AO-Core. (3) An analysis of plain-TME/SME’s confidentiality properties and of its anti-replay gap, including the specific software and TPM-counter mitigations we apply. (4) A grounded hardware-availability survey with a published methodology. (5) An honest enumeration of residual and excluded threats. LapEE invents no cryptography; its contribution is showing, carefully, that the components already exist, and that they are sufficient for the task they are being asked to do.

2 Positioning against hardware TEEs

Before the technical argument, one clarifying comparison, because the shape of this comparison drives much of what follows.

A classical hardware TEE — SGX, TDX, SEV-SNP, Apple Silicon’s Secure Enclave when acting as an iOS App Attest root — provides two things: (a) a measurement of the workload that the silicon itself produces and signs, and (b) hardware-enforced isolation of that workload from other privileged code running on the same package. LapEE delivers (a) by different means (measured boot rooted in a TPM) and declines to provide (b), because the deployment model does not require it.

TCBs are not as different as the frame suggests. It is tempting to say “hardware TEEs have a small TCB” and “LapEE has a large TCB,” but the comparison collapses under inspection. Table 1 summarizes.

System	Trusted computing base (running-workload protection)
SGX (historic)	CPU + microcode + SGX runtime + enclave code only. <i>Small.</i>
TDX CVM	CPU + microcode + SEAM module + full guest Linux (kernel, systemd, glibc, cloud-vendor guest agent, workload) . The cloud operator typically chooses the guest image; the expected measurement is provided by that same operator.
SEV-SNP CVM	CPU + microcode + PSP firmware + full guest Linux, as above . Similar operator/measurement relationship.
Apple PCC	Apple silicon + SepOS + Apple-controlled minimal guest OS, inside an Apple-controlled facility.
LapEE	CPU + memory controller + IOMMU + TPM + firmware + microcode + ME/PSP + hardened minimal Linux + HyperBEAM + devices authenticated by trusted_signers .

Table 1: Running-workload TCB comparison. SGX enclaves had a small TCB but were effectively deprecated on consumer parts. The dominant deployed hardware TEEs today (TDX CVMs, SEV-SNP CVMs) run a full guest Linux including cloud-vendor agents and drivers — often from the very operator they are meant to be confidential from. LapEE’s TCB is of the same order of magnitude; the primary delta is LapEE’s honesty about it and its use of reproducibly-built operator-signed images rather than cloud-provider-chosen ones.

Runtime integrity is shared ground. It is a fair observation that measured boot demonstrates what booted, not what the system is doing now. That observation applies to LapEE, and it applies with equal force to every deployed confidential-VM platform: once a TDX or SEV-SNP CVM has booted, a guest-kernel compromise exfiltrates as effectively as on bare metal. Runtime integrity in all of these systems rests on the kernel in the measured image being free of exploitable vulnerabilities. LapEE explicitly shares this assumption (§4, Assumption 1); it is not a LapEE-specific weakness.

What LapEE is not. LapEE does not provide co-resident-tenant isolation on a single machine. It does not provide hardware-signed measurement of the workload binary (the TPM measures via a software-driven

chain). It does not provide per-cache-line memory integrity or anti-replay in the style of SGX’s Memory Encryption Engine (and neither, in general, do plain TME, SME, or even SEV-SNP’s guest-memory encryption in the same fine-grained sense — §6.2). LapEE is an attested dedicated-appliance architecture; it is useful precisely where that architecture is useful.

3 Primitives

We assume two audiences: verifiers and implementers familiar with TPM-based attestation, who will skim this section, and decentralized-systems engineers new to these primitives, for whom it is a pointer into the normative literature.

TPM 2.0. A discrete (dTPM) or firmware (fTPM) coprocessor exposing sealed key storage, signing with non-extractable keys, 24 Platform Configuration Registers (PCRs) that accumulate **extend** operations ($\text{PCR} \leftarrow H(\text{PCR} \parallel x)$), a **quote** primitive signing PCR values plus a caller nonce, monotonic NV counters, and policy-gated key use via `TPM2_PolicyPCR` [4]. The Endorsement Key (EK) is burned at manufacture and its certificate chains to the TPM vendor’s root CA.

Measured Boot and the TCG Event Log. UEFI firmware extends hashes of each boot stage into PCRs 0–7 and emits a parallel, human-readable *event log* describing each measurement [5]. A verifier replays the log, recomputes PCR values, and checks against a TPM quote. Named fields in the log (firmware version, Secure Boot variable contents, bootloader identity, kernel hash, cmdline) are the substrate for policy rather than opaque hashes.

UEFI Secure Boot with custom keys. Stock Secure Boot trust anchors can be cleared by the machine owner and replaced by an operator-chosen Platform Key, Key Exchange Keys, and signature database [6]. Thereafter, only binaries signed by the custom chain boot. The database state is measured into PCR 7 in a well-standardized, portable way across vendors.

Total Memory Encryption. Intel TME [7] and AMD SME/TSME [8] are inline AES-XTS engines at the memory controller. A per-boot ephemeral key, retained inside the CPU package and never exposed to software, encrypts every DRAM line on the way to the DIMMs and decrypts on the way back. Above the memory controller everything is plaintext; at and past the DIMM pins everything is ciphertext. Overhead is typically < 3%. TSME is enabled unconditionally at boot; plain TME and SME require a firmware switch.

Enablement is observable by software: Intel exposes `IA32_TME_ACTIVATE` (MSR 0x982); AMD exposes `SYSCFG` bit 23 (MSR 0xC0010010). Our signed boot image reads these MSRs during its early init and refuses to proceed if the expected state is not present. Because the image is dm-verity-sealed and its UKI hash is measured into PCR 11, the fact that it reached the point of producing an attestation is itself proof that TME was active — a stronger and more portable guarantee than relying on vendor-specific firmware events. §8 documents firmware-exposure prevalence in detail.

IOMMU. VT-d (Intel) and AMD-Vi provide device-to-memory translation with per-device policy. In *strict* mode with no passthrough, peripherals cannot DMA to arbitrary physical addresses, defeating “evil peripheral” attacks over PCIe, Thunderbolt, and USB4. IOMMU enablement is controlled by kernel command-line arguments (`intel_iommu=on iommu=strict / amd_iommu=on`), which are part of the UKI and therefore covered by the PCR 11 measurement.

dm-verity and IMA. dm-verity [9] provides a Merkle tree over a read-only block device with a signed root hash, verified on every block read. IMA [10] extends each executed file’s hash into PCR 10 or 11 under policy, producing a runtime execution event log.

systemd UKI, mkosi, pcrlck. A Unified Kernel Image (UKI) is a single signed PE binary containing kernel, initramfs, and cmdline [11]. Measuring a UKI atomically extends PCR 11 with one hash over all three components — so a change to any of kernel, initrd, or cmdline produces a different PCR value. mkosi [12] builds reproducible, dm-verity-sealed, UKI-packaged minimal OS images.

AO-Core. AO-Core [2] is HyperBEAM’s computation model: every message routes through a pipeline of composable *devices*, and every execution step extends a cryptographic hashpath over its inputs and outputs. The hashpath is the same primitive as a TPM event log — a merkle chain of named, hashed events — applied at a different layer of the stack.

4 Threat Model and Trust Boundary

Actors.

Consumer C: submits work; trusted only for their own data.

Operator O: owns the LapEE machine; *adversarial by default*. Has root when not running the attested image, full physical custody, and free rein to reboot, flash firmware, attach peripherals, or swap DIMMs.

Network: any other participant routing or relaying; untrusted.

Hardware vendor: CPU + memory-controller + IOMMU manufacturer; trusted for silicon and microcode correctness. This necessarily includes the management engine (Intel ME) or platform security processor (AMD PSP), which is an indivisible part of the hardware vendor’s TCB.

TPM vendor: manufactures the TPM, signs EK certificates; trusted for chip correctness and CA hygiene. For firmware TPMs this is the same party as the hardware vendor.

HyperBEAM-OS builder: produces signed, reproducibly-built LapEE OS images and publishes source + measurements; trusted to sign only reviewed builds.

Device implementers: authors of HyperBEAM devices. At first-load of each device HyperBEAM verifies the device’s signature against `trusted_signers`; once loaded the device code is assimilated into the Erlang environment alongside the rest of the system, with no side-load path.

Adversary capabilities. The operator can: execute arbitrary code as root when booted outside the attested image; flash any firmware Secure Boot will accept; swap DIMMs, attach bus interposers, cold-boot the machine, plug in any peripheral, or refuse to run. The operator cannot: extract fuse-backed TPM keys without destructive decapping; forge a TPM vendor CA signature; decap the CPU package to recover the TME key while keeping the system running.

Assumptions.

- A1. Runtime kernel integrity.** The Linux kernel in the blessed image has no unpatched vulnerability permitting userspace to bypass IOMMU, MAC, or lockdown enforcement. Patches land promptly and old images are forced out of attestation by a verifier-enforced OS version floor. *This is the same assumption every deployed confidential computing platform makes about its guest kernel.* A TDX or SEV-SNP guest OS with a kernel 0-day leaks as effectively as bare metal; the hardware memory encryption defends the guest from the host, not the guest from itself.
- A2. TPM vendor integrity.** The TPM vendor’s signing keys are not compromised.
- A3. Supply chain integrity.** The HyperBEAM-OS build pipeline and signing keys are not compromised. Reproducible builds and multi-party signing of releases bound this.
- A4. Memory-controller correctness.** The CPU’s TME/SME key generation and XTS engine are correct.

What LapEE trusts. Only the union of trusted parties above. A consumer must trust the hardware vendor for silicon (including ME/PSP), the TPM vendor for its signing chain (the same party, for fTPM), the HyperBEAM-OS builder, and the implementers of whatever devices appear in the node’s `trusted_signers` set as reported in the attestation. The consumer does *not* trust the operator, the machine, the network, any hypervisor (none is present), or any party with physical access.

5 Architecture

5.1 Boot and measurement

Figure 1 shows the measured chain. CPU-fused root of trust (Intel Boot Guard / AMD PSP Verified Boot) anchors firmware integrity at PCR 0. UEFI Secure Boot with operator-enrolled PK/KEK/db ensures only HyperBEAM-OS-signed bootloaders and kernels load (PCR 7 records the variable contents). `systemd-boot` loads a UKI signed by HyperBEAM-OS; measuring the UKI extends PCR 11 with a single hash over kernel, `initramfs`, and `cmdline`. The kernel mounts a `dm-verity`-sealed rootfs whose Merkle root is baked

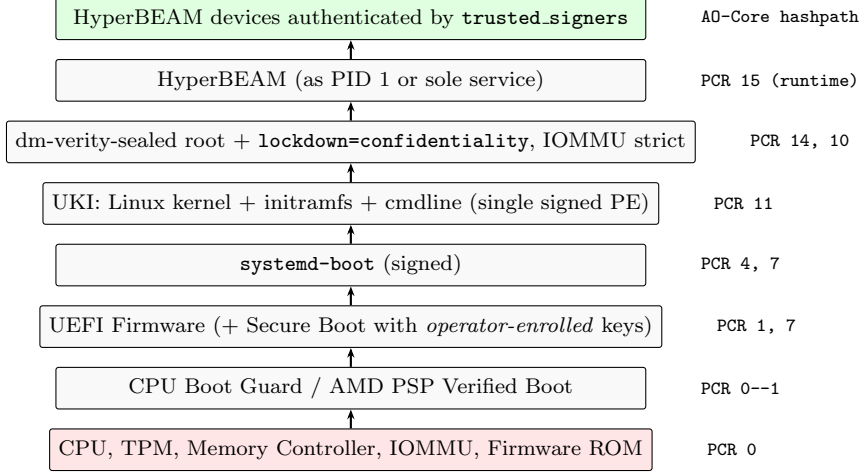


Figure 1: LapEE trust chain. Each layer is measured into a TPM PCR before execution; the TPM event log records the named event for verifier policy. Device identity at the top is authenticated against HyperBEAM’s `trusted_signers` at first-load; the AO-Core hashpath then extends the merkle chain into user-visible computation.

into the signed cmdline. Linux runs with `lockdown=confidentiality` [13], module signature enforcement, IOMMU strict mode, `init_on_alloc`, and `init_on_free`. Early init reads `IA32_TME_ACTIVATE` or `SYSCFG` and refuses to continue if memory encryption is not active. The only userspace service is HyperBEAM, launched either as PID 1 or as a single `systemd` unit.

5.2 Node key binding

At the end of the measured boot flow, HyperBEAM generates a fresh signing keypair via `TPM2.Create` under a primary key on the Endorsement hierarchy. The private key never leaves the TPM and is not made persistent: it exists only for the lifetime of this TPM context and disappears on reboot. As the *last step before attestation is generated*, HyperBEAM extends a dedicated runtime PCR (PCR 15) with the public half of the keypair. Every subsequent attestation signed by this key therefore demonstrates, cryptographically, that the key was created during this specific measured-boot session: a verifier replaying the event log sees the `key-pubkey-extend` event land into PCR 15 after all TCB measurements, at which point the PCR state is immutable for the lifetime of the boot. No attacker can produce a quote binding a different signing key to the same PCR trajectory without re-executing the entire measured boot.

Because the key is ephemeral per boot, the design avoids persistent sealed secrets. There is no long-term key material to steal, rotate, or re-seal on firmware update. Operators re-register with the network on every reboot. All TPM sessions that touch sensitive state use encrypted sessions (HMAC with parameter encryption per TPM 2.0 [4]).

5.3 Attestation evidence

An attestation consists of: (i) a `TPM2.Quote` over the operative PCR selection with a verifier-supplied nonce, signed by an Attestation Key whose chain to the EK is proved via credential activation; (ii) the TCG event log; (iii) the HyperBEAM runtime event log, including device first-load events and the terminating `key-pubkey-extend` event of §5.2; (iv) the EK certificate chain to the TPM vendor root; and (v) machine-identifying fields. Every field is a named entry in the event log rather than an opaque hash: verifiers reason over human-readable state and publish policies (“accept any node with TPM vendor in V , OS image in I , TME active, Secure Boot in *full*, IOMMU strict, device set $\subseteq D$ ”).

5.4 AO-Core continuity

AO-Core’s hashpath model [2] is structurally identical to the TPM’s extend primitive: each step is a hash over the previous chain tip and the current named event. HyperBEAM seeds its AO-Core chain with a commitment to the TPM event log tip immediately after the *pubkey-extend* event; thereafter every device first-load and every message processed extends the chain. The two logs are not analogous mechanisms doing similar work; they are the same cryptographic primitive applied at different layers of the stack, composed



Figure 2: A single merkle chain across both layers. The TPM event log extends from firmware through the terminating *pubkey-extend* event, which binds the node’s fresh signing key to the boot measurement. AO-Core’s hashpath then extends from device first-load through every subsequent message. A signature at the tip, produced by the ephemeral per-boot key whose public half is already measured into PCR 15, binds the transcript *conditional on runtime integrity of the measured image* (§4, A1): the chain proves what was booted and what the software claims it did, not that the software was not itself compromised at runtime. This conditional is structurally identical to the conditional under which TDX or SEV-SNP attestations bind their guest transcripts.

end-to-end. The caveat of Figure 2 is explicit: the chain’s integrity binds the *transcript* to the boot, and any consumer evaluating the transcript must also accept the runtime-integrity assumption that applies uniformly across this class of system.

6 Security Argument

We split the argument in two. §6.1 covers attacks *blocked at load time* by cryptographic checks and attacks *detectable in attestation* by measurement divergence. §6.2 covers runtime confidentiality and integrity of memory, including the anti-replay gap of plain TME/SME, the software mitigations we use, and the residual window where an adversary with active DRAM-write access retains some capability. §6.3 covers TPM bus sniffing.

6.1 Boot-time and attestation defenses

Table 2 enumerates attacks in two columns. “Blocked at load” means the attack cannot succeed because a cryptographic check fails before the attacker’s code runs. “Detected via attestation” means the attack *can* succeed locally on the operator’s machine but produces a measurable deviation that a verifier’s policy rejects, excluding the node from work.

Detectability is only useful when a verifier’s policy covers the property in question. LapEE exposes all attestation state as named event-log entries so that verifier policies can be written, audited, and composed (§5.2, §9): higher-layer systems — HyperBEAM’s **green-zone@1.0** admission predicate, AO-Core processes governing acceptance criteria, ad-hoc consumer-side filters — decide which subset of detectable events are load-bearing for their use case.

6.2 Memory encryption, integrity, and replay

LapEE uses plain TME on Intel and TSME on AMD. Both encrypt all DRAM with a per-boot ephemeral AES-XTS key held inside the CPU package. The tweak input is the physical address, so ciphertext cannot be relocated between addresses by an attacker with bus access. The confidentiality-at-rest guarantee is straightforward: any attacker reaching DRAM directly — cold boot, DIMM swap, passive bus interposer — recovers ciphertext only.

Plain TME and SME do not provide per-line *integrity* or *anti-replay* in the fine-grained sense of SGX’s Memory Encryption Engine (MEE), which used an on-die SRAM-backed Merkle tree to detect both tampering and ciphertext rollback [14]. This is a real gap, and it deserves the discussion below. We note the comparison class honestly: SEV-SNP does add an integrity story (the Reverse Map Table plus guest-encrypted VMSA), while TDX adds MAC-protected memory; but SEV-SNP has been the subject of published ciphertext-side-channel attacks [15], and both still rely on the guest kernel remaining honest for runtime integrity of the workload itself. The state of the art has not produced a deployed platform that closes this gap cleanly; systems in this space are comparing shades of partial.

The attack surface we retain. An attacker with *passive* access to DRAM (cold boot, DIMM removal, bus sniffing during operation) recovers ciphertext only, with no known practical analytic path to plaintext. This is the attack most commonly discussed in physical-security literature, and LapEE defeats it. An attacker with *active* DRAM-write access during operation can: (a) corrupt ciphertext at address *A*, causing

Attack	Outcome	Mechanism
Boot a modified kernel	Blocked at load	UKI signature invalid; Secure Boot refuses.
Modify initramfs or cmdline	Detected via attestation	Change recomputes UKI hash $\Rightarrow$ PCR 11 diverges.
Boot a different signed OS image	Detected via attestation	PCR 11 diverges; quote binds ephemeral key to wrong boot.
Load unsigned kernel module	Blocked at load	<code>module.sig_enforce=1</code> , kernel lockdown.
<code>/dev/kmem</code> , <code>/dev/mem</code>	Blocked at load	Not present under lockdown-confidentiality.
<code>kexec</code> to attacker kernel	Blocked at load	Disabled by lockdown.
Attach debugger to HyperBEAM	Blocked at load	Yama <code>ptrace_scope=3</code> ; kernel lockdown disallows <code>ptrace(PTRACE_ATTACH)</code> and <code>/proc/*/mem</code> .
Inject modified HyperBEAM device	Blocked at load	Fails <code>trusted_signers</code> check at first-load.
Peripheral DMA (PCIe, TB, USB4)	Blocked at load	IOMMU strict denies DMA to non-assigned pages.
Cold boot of RAM	Defense at rest	TME/SME key cleared on power loss; DIMM residue is ciphertext.
DIMM swap / bus interposer (passive)	Defense at rest	DIMM I/O is AES-XTS ciphertext.
DMA / bus replay at same address	Partial ; see §6.2	BEAM immutability, allocator randomization, TPM counters.
TPM bus sniffing (dTPM)	Blocked at load	Encrypted TPM sessions (HMAC + parameter encryption).
Enable firmware debug (DCI, JTAG)	Detected via attestation	Debug-enable bits measured into PCR 0/1.
BIOS downgrade	Detected via attestation	Firmware measurement (PCR 0) diverges; verifier version floor.
Rogue or counterfeit hardware	Blocked at load	EK cert fails to chain to known TPM vendor root.

Table 2: Attack classes by defense type. *Blocked at load* means the attack cannot locally succeed; *detected via attestation* means the node can be compromised locally but will be excluded from work by any verifier enforcing the corresponding policy; *defense at rest* means the attack on physical storage recovers ciphertext only.

the CPU to decrypt garbage on next read — likely crashing the workload rather than exfiltrating; and (b) *replay* old ciphertext at address A , causing the CPU to decrypt to a stale plaintext it genuinely produced earlier. Bus interposers with active write capability against DDR4 or DDR5 at line rate are not off-the-shelf equipment: they are lab-grade instruments in the same cost and complexity class as the chip-decapping attacks all deployed TEEs accept as out of scope.

Within this residual threat, LapEE applies three layers of mitigation against replay specifically.

(1) BEAM immutability. User-space state in the BEAM is overwhelmingly immutable. Once a term is allocated at an address, that address is not written again during its lifetime; replaying ciphertext at an unchanged address is a no-op. The narrow residual case is address reuse after garbage collection: if address A held ciphertext C_1 for a term P_1 , then A was freed and reallocated for term P_2 with ciphertext C_2 , an attacker overwriting A with C_1 causes reads to return P_1 . For this to be *useful*, P_1 must be both structurally valid at P_2 ’s semantic slot and exploitable. In practice the reused address typically holds a structurally different term whose tag bits and internal invariants do not match the stale bytes, and the BEAM detects the mismatch as memory corruption rather than accepting it as a valid term. This is a probabilistic defense, not a cryptographic one; we label it as such.

(2) Allocator randomization. To further reduce the likelihood of semantically-coherent address reuse, LapEE enables an allocation-time placement randomization — conceptually ASLR at allocation granularity — either via a hardened libc allocator or a modest BEAM patch that randomizes within the allocator’s carriers. Again probabilistic.

(3) TPM-backed counters for load-bearing invariants. For state where anti-replay must be *cryptographic*, LapEE exposes two primitives via `~tpm@2.0a`:

Volatile PCR extend: fast, resets on reboot; suitable for within-session freshness (nonces, within-process monotonic counters).

NV counter with PolicyPCR: monotonic across reboots, ≈ 10 ms per increment, incrementable only when the blessed PCR set is satisfied; suitable for durable single-assignment invariants (e.g., AO-Core slot assignments, cryptographic checkpoint heights).

The composition is: bulk state is defended probabilistically by (1) + (2) because most terms are not sensitive and the reused-address-becomes-exploit path is narrow; security-critical state (signing, nonces, monotonic invariants) is defended cryptographically by (3). Where a workload cannot tolerate even the narrow probabilistic gap, it routes the relevant state through (3).

What (1)–(3) do not cover. They apply to BEAM-managed user-space terms. State outside that scope — kernel data structures, page tables, DMA ring buffers, allocator metadata, Erlang process stacks, native-code buffers in devices — is not covered by the immutability argument. We have not identified a specific elucidated attack exploiting replay against this state; the general concern is that plain-TME offers no protection here and all defense would have to come from broader architectural properties (IOMMU strict mode for DMA metadata, kernel hardening for kernel state) or from the active-write-adversary simply being too expensive to deploy. This is a genuine limitation of LapEE, shared in qualitative terms with every plain-TME and TSME deployment.

Key management. One natural worry is whether active-DRAM-replay can compromise the node signing key. The key material itself lives in the TPM, not in DRAM, and is therefore outside the attack surface. The most an adversary could extract via replay of HyperBEAM’s user-space state is the ability to cause a re-signing of an input the node already signed. Under RSASSA-PSS with fresh randomness per signature, the resulting signature bytes differ from the original but commit to the same message; consumers of the signature care about the message’s attested content, not the specific byte sequence, so this does not break any downstream property we are aware of. We invite reviewers to construct a specific attack that disagrees; we have not found one.

6.3 TPM bus sniffing

A discrete TPM sits on an LPC or SPI bus; passive probing can extract secrets passed in cleartext sessions [16]. LapEE mandates that all sensitive TPM commands use *encrypted sessions* (HMAC with parameter encryption per TPM 2.0 [4]), which reduces passive bus observation to ciphertext. We note but deliberately do not rely on the dTPM-vs-fTPM distinction: an fTPM runs inside the ME or PSP, which is already in the hardware-vendor TCB, and if that TCB is compromised it can observe post-decryption memory state anyway, making dTPM-vs-fTPM moot. Encrypted sessions are the load-bearing mitigation; TPM type is exposed in attestation for consumers who wish to filter.

7 Residual and Excluded Threats

- **Physical attack on the CPU package.** Decapsulation, fault injection, laser probing, focused-ion-beam work on the CPU or TPM die. High-cost, destructive, per-device. Accepted as a hardware-attack-complexity bar, consistent with TDX, SEV-SNP, and Apple PCC.
- **Active-DRAM-write adversaries** (§6.2). Partially mitigated by BEAM immutability, allocator randomization, TPM counters. Residual gap against kernel and non-BEAM state.
- **Kernel 0-days.** See A1. Mitigations: signed image updates, OS version floors, minimal userspace exposure. Same assumption every deployed CC platform makes.
- **Compromise of a trusted signing party.** TPM vendor CA, HyperBEAM-OS signing key, device signing keys. Mitigations: multi-party signing, reproducible builds, publication of rotations and revocation lists.
- **Side channels against TME/SME keys.** No published full break on the plain primitive at the time of writing; TME and TSME are substantially simpler than SGX’s MEE and have attracted less research, so the absence of published attacks is weaker evidence than for older and more-studied systems.
- **Rowhammer** [17]. LapEE requires DDR4 with targeted-refresh or DDR5 with on-die ECC, recorded in attestation.
- **Liveness.** An operator can always stop. LapEE is a confidentiality and integrity story; availability is achieved by the network layer.
- **Traffic analysis.** Timing and size patterns of network-visible traffic are outside scope.

8 Hardware Availability

LapEE’s practical case depends on how much commodity hardware presents the four required primitives. This section presents grounded estimates with sources; the full methodology, SKU-selection rule, and

inclusion/exclusion detail is in Appendix A. We have not performed a formal market survey and do not claim one; the numbers below are informed estimates derived from vendor feature disclosures, Windows 11 compatibility telemetry, and industry reporting.

Class	Silicon capability	Typical BIOS exposure
Intel vPro (client, 11th gen +)	TME since Tiger Lake (2020)	~70–80%
Intel non-vPro (client, 11th gen +)	TME silicon present, patchy	~20–30%
Intel Xeon SP (server)	TME since Ice Lake-SP (2021)	~95%
AMD Ryzen Pro	TSME since Zen 2 (2019)	~70–80%
AMD Ryzen consumer	TSME silicon present, patchy	~20–40%
AMD EPYC	SME/TSME since Naples (2017)	~95%
TPM 2.0 (any)	Win 11 requirement (2021)	~100% of Win 11 capable
IOMMU (VT-d / AMD-Vi)	Standard on business-class $\geq$ 2012	~100% business-class
UEFI Secure Boot (enrollable PK)	Standard since Win 8 era (2012)	~95%

Table 3: Commodity hardware primitive availability. TME/TSME is the binding constraint; the other three primitives are near-universal on Windows-11-era business-class hardware. See Appendix A for methodology.

Composite viability and contrast. Applied to a random 2022–2023 laptop purchase, composite viability (all four primitives present and firmware-exposed) is in the 30–50% range. Filtering to business-class SKUs from the secondary market (ThinkPad T/X/P, Latitude, EliteBook) pushes viability above 70%. For contrast, TDX-capable silicon (Intel Xeon SP 4th gen +, Sapphire Rapids and later) represents a low single-digit percentage of the x86 server installed base and effectively zero percent of the client base; SEV-SNP-capable silicon (EPYC Milan +) is similarly server-only and concentrated in cloud fleets. LapEE therefore addresses an installed base larger by roughly two orders of magnitude, at per-machine costs that are lower by the same factor, and including hardware already in operators’ possession for which acquisition cost is zero.

Ancillary properties of laptops. The integrated battery provides a de-facto UPS, smoothing brief power interruptions — directly useful for liveness in a decentralized network. The widespread practice of running home-hosted LLM agents on dedicated commodity hardware (e.g., Mac minis) demonstrates the social and commercial viability of the “dedicated commodity box hosting one attested workload” pattern.

9 Implementation Sketch

A concrete LapEE node is produced by an mkosi [12] build invoked from a HyperBEAM-OS repository. The build emits: (i) a UKI signed by HyperBEAM-OS keys, containing a hardened Linux kernel with minimal driver set (network only — no USB HID, no Bluetooth, no audio, no webcam), IMA configured for execute-time measurement, and cmdline including the verity root hash and IOMMU arguments; (ii) a dm-verity-sealed rootfs containing only Erlang, HyperBEAM, and the systemd units necessary to bring up the network and start HyperBEAM; (iii) a manifest of expected PCR values computed by systemd-PCRlock [11] for publication as the golden set; and (iv) the source-tree hash for reproducible-build verification.

HyperBEAM itself gains a single new device, `~tpm@2.0a`, that wraps tpm2-tss [18]. The device name follows HyperBEAM convention (exemplified by `~lua@5.3a`): the number identifies the underlying specification version, the letter denotes the revision of the AO-Core API. `~tpm@2.0a` exposes quote, sign (using the ephemeral key of §5.2), PCR-extend, NV-read, NV-increment with PolicyPCR, and event-log-read to AO-Core messages. All sensitive operations use encrypted sessions by default; the device reports TPM manufacturer, type (fTPM or dTPM), TME/SME status, Secure Boot state, and IOMMU policy in every attestation.

LapEE is deliberately a *primitive*. It does not impose a governance model; it exposes the exact hardware and software state of a node in a form (TCG event log + AO-Core hashpath + ephemeral-key attestation) that higher-layer policies can evaluate. Concretely: HyperBEAM’s existing `green-zone@1.0` device, which maintains a shared-secret identity across cooperating nodes, can define its `is-admissible` predicate to require a valid LapEE attestation against the green zone’s chosen hardware and software constraints. Blockchain-style managed-TEE architectures, consortium admission schemes, threshold-signature committees on golden measurements, and ad-hoc consumer-side policy evaluation can all compose on top without change to LapEE.

Operator bring-up: enroll HyperBEAM-OS PK/KEK/db in firmware (one-time, via BIOS setup or tooling such as `sbctl`); boot the LapEE image from USB or internal disk; on every boot HyperBEAM generates a fresh signing key, extends its public half into PCR 15 as the final boot event, produces the initial attestation, and registers with the network under that boot’s key identity.

10 Related Work

Hardware TEEs (SGX [14], TDX [19], SEV-SNP [20]) and Apple’s Private Cloud Compute [21] frame the design space. SGX’s enclave model provided workload-identity-bound keys — a primitive LapEE does not replicate — but was deprecated on consumer Intel parts, and its design has been repeatedly eroded by speculative-execution attacks [22]. TDX and SEV-SNP shifted abstraction to confidential VMs running full guest OSes. That shift substantially enlarged the running-workload TCB (Table 1) and exposed ciphertext-channel attack surface [15]; it also, in the dominant cloud deployment pattern, reintroduces the cloud operator as an expected-measurement-setting party, which narrows the effective trust improvement in practice considerably. LapEE’s TCB, honestly tallied, is not materially larger than a deployed CVM’s, and its expected measurements are set by reproducibly-buildable operator-signed code rather than by the same party the deployment is meant to be confidential from.

Apple’s PCC [21] is architectural precedent for “minimal attested appliance, physical security by facility controls.” PCC assumes Apple-owned hardware and an Apple-controlled facility; LapEE drops both assumptions and substitutes measured boot with operator-enrolled trust anchors.

A recent proposal targeting Apple Silicon [3] applies software-access-path elimination to a hardware substrate that exposes no third-party-accessible measurement register, leaving the “running binary” field of attestation self-reported by the app itself. LapEE avoids this gap by rooting measurement in a commodity TPM where app-equivalent state (UKI hash, rootfs Merkle root, HyperBEAM binary hash, loaded device hashes) is extended into PCRs by code the attacker cannot replace while maintaining attestation.

Measured-boot attestation infrastructure for cloud and enterprise is mature: Keylime [23] implements verifier-side policy evaluation; systemd-PCRlock [11] produces golden-value artifacts; RATS [24] gives the terminology. Prior work on CPU-resident keys (TRESOR [25], Copker [26]) informs the replay discussion. Flashbots’ BuilderNet [27] demonstrates TDX-attested decentralized compute but trusts the cloud operator’s expected measurement, the circular-trust problem LapEE avoids by operator-enrolled Secure Boot and open golden-measurement publication. Multi-node coordination is not a LapEE addition: AO-Core is already a hyper-parallel decentralized compute substrate [2]; a mesh of LapEE nodes is simply a mesh of AO-Core nodes that happen to present attestable trust.

11 Conclusion

The public narrative in decentralized compute has converged on the idea that meaningful hardware-rooted trust requires a classical hardware TEE — TDX, SEV-SNP, or equivalent — and therefore requires silicon priced for the cloud. This paper argues the premise is wrong for the decentralized-compute threat model specifically. Four primitives already shipping in commodity laptops and desktops — TPM 2.0, UEFI Secure Boot with operator-enrolled keys, IOMMU, and platform memory encryption — suffice to produce an attested dedicated appliance whose trusted computing base is not materially larger than a deployed confidential VM’s, whose per-machine cost is one to two orders of magnitude lower, and whose installed base is larger by roughly the same factor. The trade is explicit and single-machine: no co-resident tenants. In a network that provides multi-tenancy by distributing work across machines, that is the correct trade to make. LapEE is a primitive that exposes exactly the hardware and software state a consumer needs to evaluate, composing cleanly with HyperBEAM’s device model and AO-Core’s hashpath continuity. Older hardware already in operators’ possession can serve as LapEE nodes at zero acquisition cost. The take-home message: for decentralized compute, a TEE in the classical sense is not a necessary ingredient. An attested dedicated appliance built on commodity primitives, running code anyone can audit and reproduce, is sufficient — and available now.

Availability. Source, reproducible build recipes, golden-measurement publication, and the `~tpm@2.0a` device implementation are being prepared for release at github.com/permaweb/hb-os (bare-metal branch).

References

- [1] Forward Research. *HyperBEAM: A decentralized node implementation for AO-Core*. 2024. URL: <https://github.com/permaweb/HyperBEAM>.
- [2] Sam Williams and Forward Research. *AO-Core: Hyper-Parallel Decentralized Computing on the Permaweb*. 2024. URL: <https://ao.ar.io/>.
- [3] Gajesh Naik. *Private Decentralized Inference on Consumer Hardware: Enabling Confidential Computation on Third-Party Apple Silicon via Software Access Path Elimination*. Tech. rep. Eigen Labs, 2026.
- [4] Trusted Computing Group. *TPM 2.0 Library Specification, Family “2.0”, Level 00, Revision 01.59*. 2019. URL: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>.
- [5] Trusted Computing Group. *TCG PC Client Platform Firmware Profile Specification, Version 1.05 Revision 23*. 2021. URL: <https://trustedcomputinggroup.org/resource/pc-client-specific-platform-firmware-profile-specification/>.
- [6] UEFI Forum. *Unified Extensible Firmware Interface (UEFI) Specification, Version 2.10*. 2022. URL: <https://uefi.org/specifications>.
- [7] Intel Corporation. *Intel Architecture Memory Encryption Technologies Specification, Rev 1.4*. 2022. URL: <https://cdrdv2.intel.com/v1/dl/getContent/679154>.
- [8] David Kaplan, Jeremy Powell, and Tom Woller. *AMD Memory Encryption*. AMD Whitepaper. 2016. URL: <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/memory-encryption-white-paper.pdf>.
- [9] The Linux Kernel Organization. *dm-verity: Device-mapper block integrity checking target*. 2024. URL: <https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/verity.html>.
- [10] The Linux Kernel Organization. *Integrity Measurement Architecture (IMA)*. 2024. URL: <https://sourceforge.net/p/linux-ima/wiki/Home/>.
- [11] Lennart Poettering. *systemd-PCRlock(8) and “Brave New Trusted Boot World”*. 2023. URL: <https://0pointer.net/blog/brave-new-trusted-boot-world.html>.
- [12] systemd Project. *mkosi — Build Bespoke OS Images*. 2024. URL: <https://github.com/systemd/mkosi>.
- [13] The Linux Kernel Organization. *Kernel lockdown: confidentiality and integrity modes*. URL: <https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/index.html>.
- [14] Victor Costan and Srinivas Devadas. “Intel SGX Explained”. In: *IACR Cryptology ePrint Archive, Report 2016/086*. 2016.
- [15] Mengyuan Li et al. “CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel”. In: *USENIX Security*. 2021.
- [16] Denis Andzakovic. “Extracting BitLocker keys from a TPM (PulseSecurity Blog and DEFCON talks on TPM bus sniffing)”. In: *PulseSecurity Blog / DEFCON*. 2021.
- [17] Yoongu Kim et al. “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors”. In: *ISCA*. 2014.
- [18] TPM2 Software Community. *tpm2-tss: OSS implementation of the TCG TPM2 Software Stack*. 2024. URL: <https://github.com/tpm2-software/tpm2-tss>.
- [19] Intel Corporation. *Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification*. 2023. URL: <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>.
- [20] Advanced Micro Devices. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Whitepaper. 2020.
- [21] Apple Inc. *Blog: Private Cloud Compute: A new frontier for AI privacy in the cloud*. 2024. URL: <https://security.apple.com/blog/private-cloud-compute/>.
- [22] Jo Van Bulck et al. “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution”. In: *USENIX Security*. 2018.
- [23] Nabil Schear et al. “Bootstrapping and Maintaining Trust in the Cloud”. In: *ACSAC*. 2016.

- [24] H. Birkholz et al. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334, IETF. 2023. URL: <https://datatracker.ietf.org/doc/rfc9334/>.
- [25] Tilo Müller, Felix C. Freiling, and Andreas Dewald. “TRESOR Runs Encryption Securely Outside RAM”. In: *USENIX Security*. 2011.
- [26] Le Guan et al. “Copker: Computing with Private Keys without RAM”. In: *NDSS*. 2014.
- [27] Flashbots. *BuilderNet: a Decentralised Block Building Network for Ethereum*. 2024. URL: <https://buildernet.org/>.

A Hardware-Availability Methodology

The percentages in Table 3 are informed estimates, not measured market-survey data. This appendix documents how we derived them.

TPM 2.0, IOMMU, Secure Boot. We treat these as near-universal on Windows-11-era business-class hardware because they are Windows 11 system requirements published by Microsoft. Windows 11 reached substantial desktop market share by 2023. Machines marketed as Windows 11 compatible by their vendors necessarily ship with TPM 2.0 enabled, Secure Boot enabled, and platform features (including IOMMU) configured for the OS. We rely on Microsoft’s published requirements documentation as the source for these claims and do not further survey the installed base.

TME/TSME silicon capability. Intel product briefs and datasheets for 11th-gen Core (Tiger Lake, 2020) and later client SKUs document TME capability in the memory controller. AMD product documentation for Zen 2 (2019) and later Ryzen SKUs documents SME/TSME capability in the IOD. We treat these vendor publications as authoritative for “capability present in silicon.”

BIOS exposure (the binding constraint). Whether TME or TSME is actually enabled at boot depends on vendor BIOS implementation. No vendor publishes this systematically. We estimate exposure rates by:

1. Surveying the **vPro**-marketed portion of Intel client SKUs (Intel publishes SKU-level feature tables): vPro implies enterprise security features are exposed, and TME exposure is strongly correlated. We assume $\sim 70\text{--}80\%$ exposure in this segment.
2. Surveying **Ryzen Pro**-marketed AMD SKUs against AMD’s Pro feature disclosures, which similarly imply exposure of platform security features. We assume $\sim 70\text{--}80\%$.
3. For non-Pro consumer SKUs, we rely on community-maintained compatibility lists and vendor-specific BIOS release notes, which show wide variability and incomplete coverage. We assume $\sim 20\text{--}40\%$ as a range rather than a point estimate.
4. For server SKUs (Xeon SP, EPYC), we assume $\sim 95\%$ exposure because server BIOS offerings reliably expose such features for cloud and datacenter customers.

Composite viability. The composite “all four primitives present and exposed” figure is the product of the individual per-feature rates, not a directly measured quantity. Composite for random 2022–2023 laptop sales: roughly $0.5 \times 0.25 \times 0.95 \times 0.95 \approx 0.11$ using the lower bounds and ≈ 0.35 using the upper bounds; rounding to $\sim 30\text{--}50\%$ for the typical case reflects that laptop *sales* skew business-class more than the installed base. For business-class secondary market machines explicitly filtered for vPro / Ryzen Pro, the composite approaches $\sim 70\%$ as per-feature rates cluster in the $\sim 90\%+$ range conditional on that filter.

Contrast set sizes. Intel 4th-gen Xeon SP with TDX (Sapphire Rapids, 2023) and later units account for low single-digit percent of the x86 server installed base (industry estimates of *new* server shipments in 2023–2024 suggest $\sim 5\text{--}10\%$; installed base is considerably lower). SEV-SNP-capable EPYC (Milan and later) is similarly concentrated in cloud fleets. Client CPUs with CVM-style TEEs do not meaningfully exist.

Limitations. We have not: built a SKU-level database; issued compatibility attestations across a population of tested machines; or measured the installed base directly. Readers should treat Table 3 and the surrounding text as reasonable engineering estimates for the purpose of LapEE’s deployment feasibility argument, not as empirical market research. A formal survey would be valuable future work; we do not condition the paper’s conclusions on it.